МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР (системы автоматизированного проектирования)

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Алгоритмы и структуры данных»

Тема: Сжатие изображения

Студент гр. 4351 _____ Погодин Д. А.

Преподаватель _____ Пестерев Д.О.

Санкт-Петербург

2025

Задание:

- Для каждого изображения из тестовых данных построить график зависимости размера сжатого изображения от параметра качества (шаг параметра качества не более 10).
- Произвести декомпрессию сжатых изображений из предыдущего пункта и приложить получившиеся изображения.

Теоретическая часть

JPEG-инспирированный алгоритм – алгоритм, который не является строгим стандартом JPEG, но использует те же ключевые идеи и этапы обработки изображения. Алгоритм состоит из нескольких основных пунктов:

1. **Цветовое преобразование:**

Изображение для удобства переводят из RGB в YCbCr, где :

Y – яркость

Cb, Cr – цветоразностные компоненты.

Это требуется из-за того, что глаз более чувствителен к яркости, чем к цвету. Поэтому, отделив яркость от цвета, получиться сжать изображение, чтобы сильно не исказить изображение.

Временная сложность:

Мы проходим по всем пикселям изображения и переводим RGB в YCbCr, значит можно сказать, что сложность будет равна $\theta(n)$, где n – количество пикселей в изображении.

Пространственная сложность:

Нужно хранить информацию о цвете о каждом пикселе, значит сложность будет также равна $\theta(n)$.

2. **Разбиение на блоки:**

Разбиваем изображение на блоки 8 на 8 для лучшей обработки изображения.

Временная сложность:

Каждый пиксель записывается в блоки, значит считаем сложность также $\theta(n)$.

Пространственная сложность:

Создаются блоки и в них добавляются в них пиксели, значит точно также сложность равна $\theta(n)$.

3. Дискретное косинусное преобразование и квантование:

Эти операции происходят в созданных блоках и позволяет оставить в них в основном плавные переходы цветов без шума и других искажений.

Временная сложность:

Так как блок всегда одного размера, значит в самом блоке сложность можно считать $\theta(1)$, а полная сложность будет $\theta(a)$, где а – количество блоков ($\approx \frac{n}{64}$).

Пространственная сложность:

Точно также обрабатывается каждый блок, значит сложность будет также $\theta(a)$.

4. Зигзагообразный обход:

Обход осуществляется также в фиксированных блоках, делая из матрицы последовательность для дальнейшего сжатия.

Временная сложность:

Точно также, как в дискретном косинусном преобразовании и квантовании и равна $\theta(a)$.

Пространственная сложность:

Точно также, как в дискретном косинусном преобразовании и квантовании и равна $\theta(a)$.

5. RLE кодирование и Хаффман:

Кодирование происходит в также в блоках, что значит временная и пространственная сложность также будет равна $\theta(a)$.

6. Восстановление изображения:

Восстанавливает получившееся изображение. Все также выходит в количество блоков в изображении, что также дает на сложность $\theta(a)$.

Практическая часть.

Тестовые данные:

- Lena.png (512x512 px)
- Цветное изображение размером не менее 512x512 px (bm1v8ul3ob191.png)
- Изображение в оттенках серого, полученное путем конвертации цветного изображения (gray.png)
- Черное-белое изображение, полученное путем округления оттенков серого (bw.png)
- Черное-белое изображение, сконвертированное из цветного/оттенков серого с использованием дизеринга (bwdizer.png)

Рис. 1 – Lena.png

Рис. 2 – bm1v8ul3ob191.png

Рис. 3 – gray.png

Рис. 4 – bw.png

Рис. 5 – bwdizer.png

Требуется пропустить каждое изображение пропустить через компрессор и вывести графики зависимости размера изображения от качества.

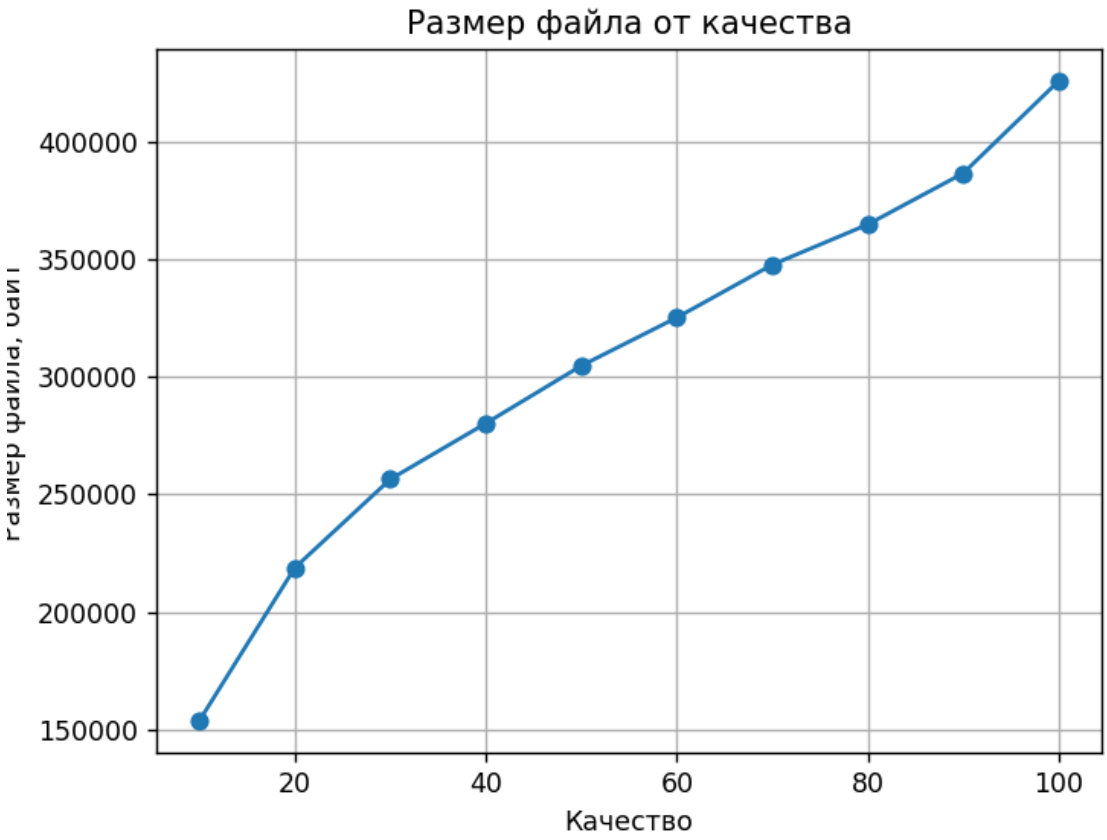

Рис. 6 – график зависимости размера файла Lena.png (в байтах) от качества

Рис. 7 – изображение при качестве 10 и 100 соответственно

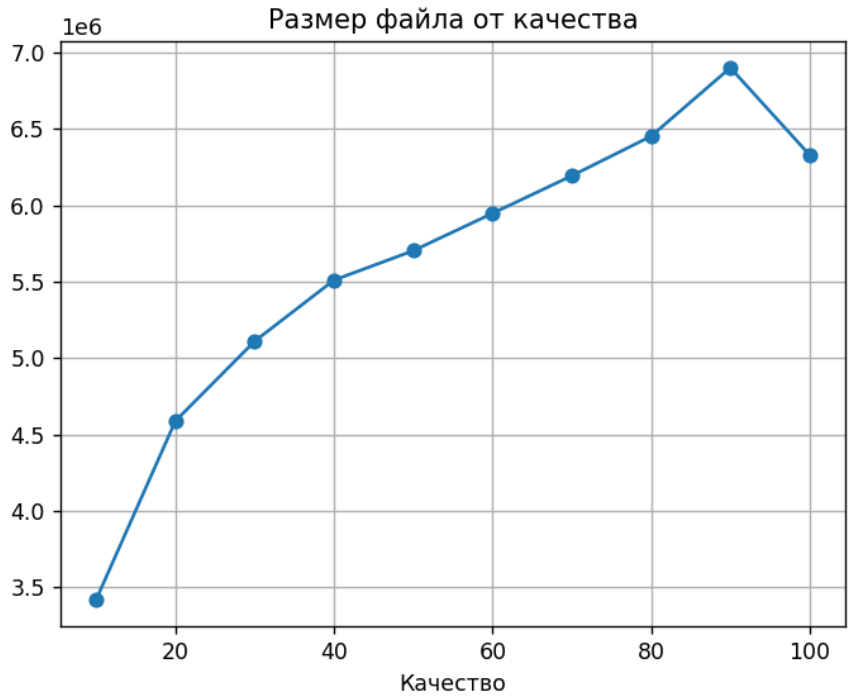

Рис. 8 – график зависимости размера файла bm1v8ul3ob191.png (в мегабайтах) от качества

Рис. 9 – изображение при качестве 10 и 100 соответственно

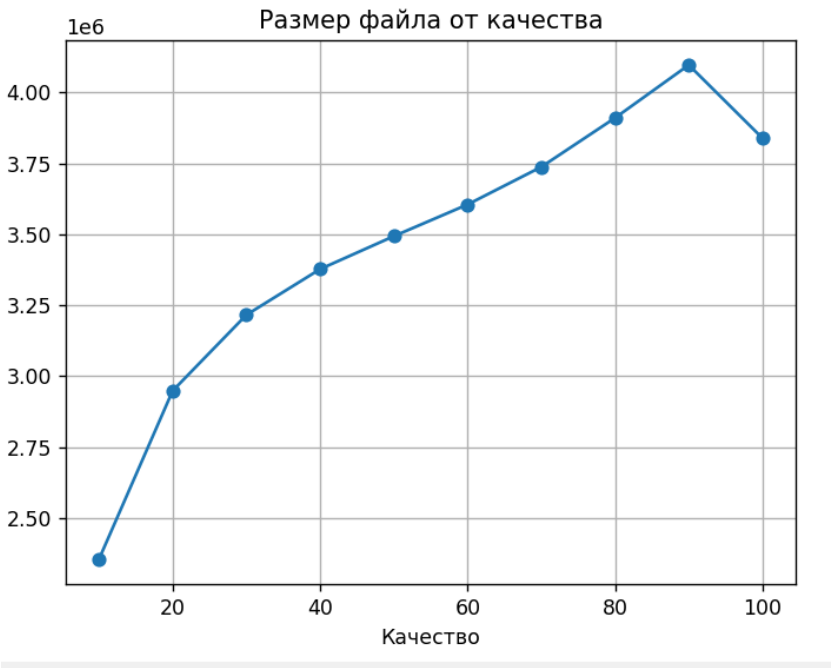

Рис. 10 – график зависимости размера файла gray.png (в мегабайтах) от качества

Рис. 11 – изображение при качестве 10 и 100 соответственно

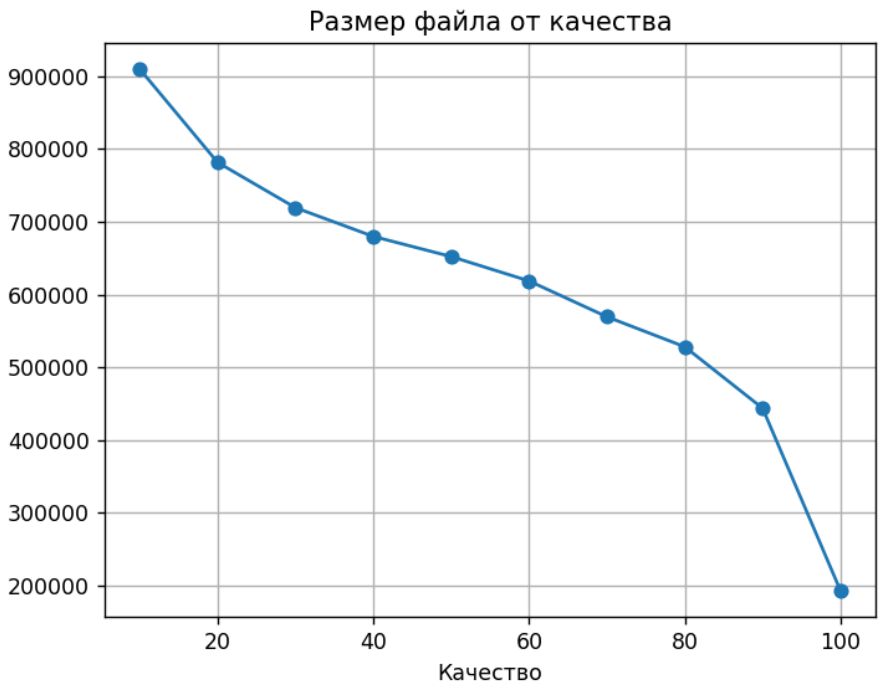

Рис. 12 – график зависимости размера файла bw.png (в байтах) от качества

Рис. 13 – изображение при качестве 10 и 100 соответственно

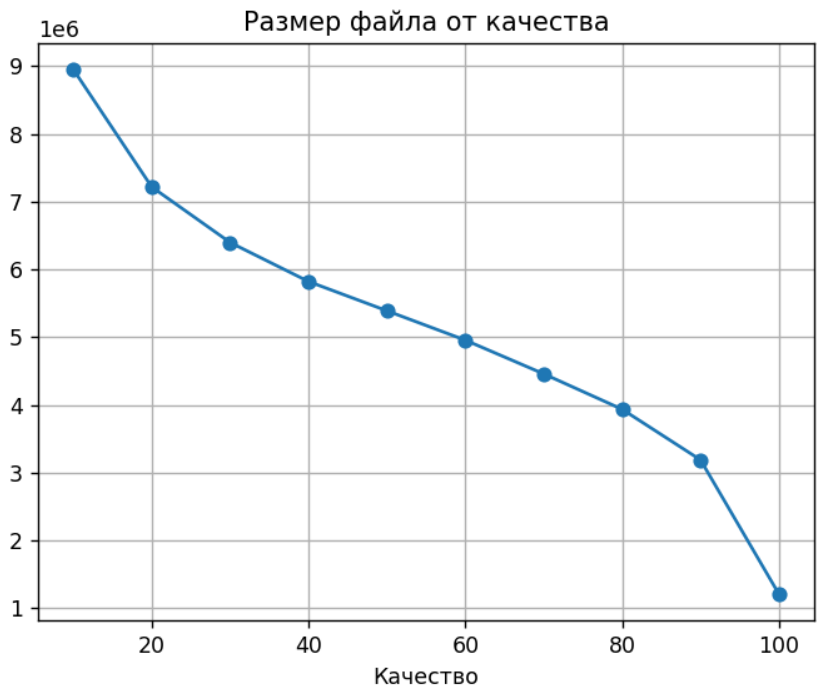

Рис. 14 – график зависимости размера файла bwdizer.png (в мегабайтах) от качества

Рис. 15 – изображение при качестве 10 и 100 соответственно

Большая часть изображений (Lena.png, цветное, оттенки серого) по графику видно, что чем больше качество, тем больше размер файла. Но с оставшимися (черно-белое, ч/б, сконвертированное из цветного/оттенков серого с использованием дизеринга) происходит все с точностью да наоборот, от роста качества понижается размер файла. Это могло произойти из-за того, что при каждом новом качестве строится новый хаффман, что могло повлиять на размер файла.

Ссылка на репозиторий: https://github.com/spg1d6/AISD_lab2-sem4-

Код программы.

```
import numpy as np
import os
from PIL import Image
import matplotlib.pyplot as plt
import pickle
import heapq
from collections import defaultdict
import struct

def bits_to_bytes(bitstring):
    padding = 8 - len(bitstring) % 8 if len(bitstring) % 8 != 0 else 0
    bitstring += "0" * padding
    b = bytearray()
    for i in range(0, len(bitstring), 8):
        b.append(int(bitstring[i:i+8], 2))
    return bytes(b), padding

def bytes_to_bits(data, padding):
    bitstring = ''.join(f'{byte:08b}' for byte in data)
    if padding:
        return bitstring[:-padding]
    return bitstring

class HuffmanNode:
    def __init__(self, value=None, freq=0, left=None, right=None):
        self.value = value
        self.freq = freq
        self.left = left
        self.right = right

    def __lt__(self, other):
        return self.freq < other.freq


def build_huffman_table(data_list):
    freq = defaultdict(int)
    for item in data_list:
        freq[item] += 1

    heap = [HuffmanNode(val, frq) for val, frq in freq.items()]
```

```
    heapq.heapify(heap)

    if not heap:
        return {}

    while len(heap) > 1:
        l = heapq.heappop(heap)
        r = heapq.heappop(heap)
        heapq.heappush(heap, HuffmanNode(None, l.freq + r.freq, l, r))

    root = heap[0]
    codes = {}

    def dfs(node, code=""):
        if node.value is not None:
            codes[node.value] = code or "0"
        else:
            dfs(node.left, code + "0")
            dfs(node.right, code + "1")

    dfs(root)
    return codes


def encode_huffman(data, table):
    bitstring = ".join(table[v] for v in data)
    return bits_to_bytes(bitstring)


def decode_huffman(data_bytes, table, padding):
    bitstring = bytes_to_bits(data_bytes, padding)
    reverse = {v: k for k, v in table.items()}

    res = []
    cur = ""
    for b in bitstring:
        cur += b
        if cur in reverse:
            res.append(reverse[cur])
            cur = ""
    return res
```

```
def split_blocks(ch, size=8):
    h, w = ch.shape
    pad_h = (size - h % size) % size
    pad_w = (size - w % size) % size
    ch = np.pad(ch, ((0, pad_h), (0, pad_w)))

    blocks = []
    for i in range(0, ch.shape[0], size):
        for j in range(0, ch.shape[1], size):
            blocks.append(ch[i:i+size, j:j+size])
    return np.array(blocks), ch.shape


def merge_blocks(blocks, shape, size=8):
    h, w = shape
    img = np.zeros((h, w))
    k = 0
    for i in range(0, h, size):
        for j in range(0, w, size):
            img[i:i+size, j:j+size] = blocks[k]
            k += 1
    return img

def create_dct(N=8):
    C = np.zeros((N, N))
    for i in range(N):
        for j in range(N):
            a = 1/np.sqrt(2) if j == 0 else 1
            C[i, j] = a * np.cos((2*i+1)*j*np.pi/(2*N))
    return C * np.sqrt(2/N)


def dct2(b, C): return C.T @ b @ C
def idct2(b, C): return C @ b @ C.T

QY = np.array([
 [16,11,10,16,24,40,51,61],
 [12,12,14,19,26,58,60,55],
 [14,13,16,24,40,57,69,56],
 [14,17,22,29,51,87,80,62],
 [18,22,37,56,68,109,103,77],
 [24,35,55,64,81,104,113,92],
```

```
 [49,64,78,87,103,121,120,101],
 [72,92,95,98,112,100,103,99]
])

QC = np.array([
 [17,18,24,47,99,99,99,99],
 [18,21,26,66,99,99,99,99],
 [24,26,56,99,99,99,99,99],
 [47,66,99,99,99,99,99,99],
 [99,99,99,99,99,99,99,99],
 [99,99,99,99,99,99,99,99],
 [99,99,99,99,99,99,99,99],
 [99,99,99,99,99,99,99,99]
])


def scale_q(Q, q):
    S = 5000/q if q < 50 else 200 - 2*q
    Qn = np.floor((Q*S + 50)/100)
    Qn[Qn == 0] = 1
    return Qn

def zigzag(b):
    res = []
    for s in range(15):
        for i in range(s+1):
            j = s - i
            if i < 8 and j < 8:
                res.append(b[i, j] if s % 2 == 0 else b[j, i])
    return res


def inv_zigzag(v):
    b = np.zeros((8, 8))
    k = 0
    for s in range(15):
        for i in range(s+1):
            j = s - i
            if i < 8 and j < 8:
                if s % 2 == 0:
                    b[i, j] = v[k]
                else:
```

```
                b[j, i] = v[k]
            k += 1
    return b


def dc_diff(dc):
    return [dc[0]] + [dc[i]-dc[i-1] for i in range(1, len(dc))]


def dc_restore(dc):
    res = [dc[0]]
    for i in range(1, len(dc)):
        res.append(res[-1] + dc[i])
    return res


def rle(ac):
    res = []
    z = 0
    for v in ac:
        if v == 0:
            z += 1
        else:
            res.append((z, v))
            z = 0
    res.append((0, 0))
    return res


def rle_decode(data):
    res = []
    for z, v in data:
        if (z, v) == (0, 0):
            res += [0] * (63 - len(res))
            break
        res += [0]*z + [v]
    return res[:63]


def rgb_to_ycbcr(img):
    img = img.astype(np.float32)
    Y = 0.299*img[...,0] + 0.587*img[...,1] + 0.114*img[...,2]
```

```
    Cb = -0.1687*img[...,0] - 0.3313*img[...,1] + 0.5*img[...,2] + 128
    Cr = 0.5*img[...,0] - 0.4187*img[...,1] - 0.0813*img[...,2] + 128
    return Y, Cb, Cr


def ycbcr_to_rgb(Y, Cb, Cr):
    R = Y + 1.402*(Cr-128)
    G = Y - 0.34414*(Cb-128) - 0.71414*(Cr-128)
    B = Y + 1.772*(Cb-128)
    return np.clip(np.stack((R,G,B), axis=-1),0,255).astype(np.uint8)


def downsample(ch):
    h, w = ch.shape
    if h % 2 != 0:
        ch = np.pad(ch, ((0,1),(0,0)), mode='edge')
    if w % 2 != 0:
        ch = np.pad(ch, ((0,0),(0,1)), mode='edge')

    return (ch[0::2,0::2] +
            ch[1::2,0::2] +
            ch[0::2,1::2] +
            ch[1::2,1::2]) / 4


def upsample(ch, shape):
    up = np.repeat(np.repeat(ch,2,0),2,1)
    return up[:shape[0], :shape[1]]

def compress(img, q):
    C = create_dct()
    orig_shape = img.shape[:2]

    Y, Cb, Cr = rgb_to_ycbcr(img)
    Cb, Cr = downsample(Cb), downsample(Cr)

    def proc(ch, Q):
        blocks, shape = split_blocks(ch)
        dc, ac = [], []
        for b in blocks:
            zz = zigzag(np.round(dct2(b-128, C)/Q))
            dc.append(int(zz[0]))
```

```
            ac += rle(zz[1:])
        return dc, ac, shape

    Ydc, Yac, ShY = proc(Y, scale_q(QY, q))
    Cbdc, Cbac, ShC = proc(Cb, scale_q(QC, q))
    Crdc, Crac, _ = proc(Cr, scale_q(QC, q))

    Ydc, Cbdc, Crdc = dc_diff(Ydc), dc_diff(Cbdc), dc_diff(Crdc)

    dc_all = Ydc + Cbdc + Crdc
    ac_all = Yac + Cbac + Crac

    dc_table = build_huffman_table(dc_all)
    ac_table = build_huffman_table(ac_all)

    dc_bytes, dc_pad = encode_huffman(dc_all, dc_table)
    ac_bytes, ac_pad = encode_huffman(ac_all, ac_table)

    return dict(
        dc_bits=dc_bytes,
        ac_bits=ac_bytes,
        dc_table=dc_table,
        ac_table=ac_table,
        dc_pad=dc_pad,
        ac_pad=ac_pad,
        shapes=(ShY, ShC),
        counts=(len(Ydc), len(Cbdc), len(Crdc)),
        q=q,
        orig_shape=orig_shape
    )


def decompress(data):
    C = create_dct()

    dc = decode_huffman(data["dc_bits"], data["dc_table"], data["dc_pad"])
    ac = decode_huffman(data["ac_bits"], data["ac_table"], data["ac_pad"])

    Yc, Cbc, Crc = data["counts"]
    ShY, ShC = data["shapes"]

    Ydc = dc_restore(dc[:Yc])
```

```
    Cbdc = dc_restore(dc[Yc:Yc+Cbc])
    Crdc = dc_restore(dc[Yc+Cbc:])

    def rebuild(dc, shape, Q, idx):
        blocks = []
        for d in dc:
            r = []
            while True:
                p = ac[idx]
                idx += 1
                r.append(p)
                if p == (0, 0):
                    break
            zz = [d] + rle_decode(r)
            block = idct2(inv_zigzag(zz)*Q, C) + 128
            blocks.append(np.clip(block, 0, 255))
        return merge_blocks(blocks, shape), idx

    idx = 0
    Y, idx = rebuild(Ydc, ShY, scale_q(QY, data["q"]), idx)
    Cb, idx = rebuild(Cbdc, ShC, scale_q(QC, data["q"]), idx)
    Cr, idx = rebuild(Crdc, ShC, scale_q(QC, data["q"]), idx)

    Cb = upsample(Cb, Y.shape)
    Cr = upsample(Cr, Y.shape)

    rgb = ycbcr_to_rgb(Y, Cb, Cr)

    h, w = data["orig_shape"]
    return rgb[:h, :w]

def save_custom_raw(image_path, raw_path):
    img = np.array(Image.open(image_path).convert("RGB"), dtype=np.uint8)
    h, w = img.shape[:2]
    header = struct.pack("<4sII", b'RIMG', w, h)
    with open(raw_path, "wb") as f:
        f.write(header)
        f.write(img.tobytes())


def load_custom_raw(raw_path):
    with open(raw_path, "rb") as f:
```

```
        magic, w, h = struct.unpack("<4sII", f.read(12))
        data = f.read(w*h*3)
        return np.frombuffer(data, dtype=np.uint8).reshape((h, w, 3))


def run(image_path):
    img = np.array(Image.open(image_path).convert("RGB"))

    qs = range(10, 101, 10)
    sizes = []

    os.makedirs("results", exist_ok=True)

    for q in qs:
        data = compress(img, q)

        restored = decompress(data)
        restored = np.clip(restored, 0, 255).astype(np.uint8)

        out_path = os.path.join("results", f"out_q{q}.png")

        img_pil = Image.fromarray(restored, mode="RGB")
        img_pil.save(out_path, format="PNG")

        size = os.path.getsize(out_path)
        sizes.append(size)

    plt.plot(list(qs), sizes, 'o-')
    plt.title("Размер файла от качества")
    plt.xlabel("Качество")
    plt.ylabel("Размер файла, байт")
    plt.grid()
    plt.show()


#run(r"D:\Users\User\Desktop\тоэ\Lenna.png")
#run(r"D:\Users\User\Desktop\тоэ\bm1v8ul3ob191.png")
#run(r"D:\Users\User\Desktop\тоэ\gray.png")
#run(r"D:\Users\User\Desktop\тоэ\bw.png")
#run(r"D:\Users\User\Desktop\тоэ\bwdizer.png")
```